DOM-based vulnerabilities (XSS)

ℹ️ Introduction

What is DOM-based vulnerability? What is the impact of a successful DOM-based attack? How do DOM-based vulnerabilities arise in web applications? How to prevent DOM-based vulnerabilities?

What is DOM-Based Vulnerability?

The Document Object Model (DOM) is a web browser's hierarchical representation of the elements on a page. Websites use JavaScript to manipulate the nodes and objects of the DOM, as well as their properties. DOM-based vulnerabilities arise when a website contains JavaScript that takes an attacker-controllable value, known as a **source**, and passes it into a dangerous function, known as a **sink**.

Two key concepts are important for understanding DOM-based vulnerabilities:

- **Source** - a JavaScript property that accepts data potentially controlled by an attacker. Common sources include `location.search`, `document.referrer`, `document.cookie`, and web messages. Essentially, any property that an attacker can influence is a potential source.
- **Sink** - a potentially dangerous JavaScript function or DOM object that causes undesirable effects if attacker-controlled data is passed to it. For example, `eval()` executes its argument as JavaScript, and `document.body.innerHTML` allows injection of arbitrary HTML.

DOM-based vulnerabilities arise when data flows from a source to a sink, and the sink handles that data in an unsafe way in the context of the client's session. This is known as **taint flow**.

What is the Impact of a Successful DOM-Based Attack?

The impact depends on which sink the attacker-controlled data reaches. Common attack outcomes include:

- **DOM XSS** - malicious JavaScript is injected and executed in the victim's browser via sinks like `document.write()` or `innerHTML`, allowing account takeover, cookie theft, or malicious actions on behalf of the user.
- **Open redirection** - the victim is silently redirected to a malicious website via sinks like `window.location`, enabling phishing attacks.
- **Cookie manipulation** - session cookies are modified via the `document.cookie` sink, potentially leading to session fixation or other attacks.
- **JavaScript injection** - arbitrary JavaScript is executed through sinks like `eval()`, giving the attacker full control over the client-side behavior.
- **DOM clobbering** - an attacker injects HTML that overwrites global JavaScript variables, changing the behavior of the application in ways that can lead to XSS or other exploits.

DOM-Based Vulnerabilities in Different Contexts

Several types of software flaws allow DOM-based attacks to succeed:

Unsafe handling of URL data - JavaScript reads attacker-controlled values from sources like `location.hash`, `location.search`, or `document.referrer` and passes them directly to dangerous sinks without validation. For example, reading a URL fragment and assigning it to `window.location` can cause an open redirect.

DOM XSS via innerHTML or document.write() - user-controlled input is inserted directly into the DOM using sinks that interpret HTML, allowing injection of malicious scripts that execute in the victim's browser.

Cookie manipulation - JavaScript writes attacker-controlled data to `document.cookie`, allowing the attacker to overwrite session-related cookies and potentially fix the victim's session to one they control.

JavaScript injection via eval() - attacker-controlled data is passed to functions like `eval()`, `setTimeout()`, or `setInterval()` which execute their argument as code, enabling arbitrary JavaScript execution.

Web message vulnerabilities - the application listens for messages via `postMessage()` but does not verify the origin of the message. An attacker can send a malicious message from another window or iframe that gets processed as trusted data.

DOM clobbering - an attacker injects HTML elements with specific `id` or `name` attributes that overwrite global JavaScript variables used by the application, altering its logic in an exploitable way.

How to Prevent DOM-Based Vulnerabilities

DOM-based vulnerabilities can be prevented by applying the following principles:

- **Avoid passing untrusted data to dangerous sinks** - the most effective defense is to never allow data from any untrusted source to dynamically alter the value passed to a sink such as `eval()`, `innerHTML`, or `window.location`.
- **Validate input on a whitelist basis** - where dynamic behavior is unavoidable, only allow content that is explicitly known to be safe. Reject anything that does not match the expected format.
- **Sanitize and encode data appropriately** - depending on the context, apply the correct combination of JavaScript escaping, HTML encoding, and URL encoding before inserting data into the DOM.
- **Verify message origin in postMessage handlers** - always check the `event.origin` property before processing any data received via web messages, and reject messages from untrusted origins.
- **Avoid dangerous sinks where possible** - prefer safer alternatives such as `textContent` instead of `innerHTML`, and `window.location.assign()` with validated URLs instead of directly assigning to `window.location`.
- **Use a Content Security Policy (CSP)** - a well-configured CSP can limit the damage of DOM XSS by restricting which scripts are allowed to execute on the page.

> *The labs presented in the following section focus specifically on DOM-based Cross-Site Scripting (XSS) vulnerabilities, demonstrating how attacker-controlled data reaches sinks that execute or render HTML and JavaScript in the victim's browser.*

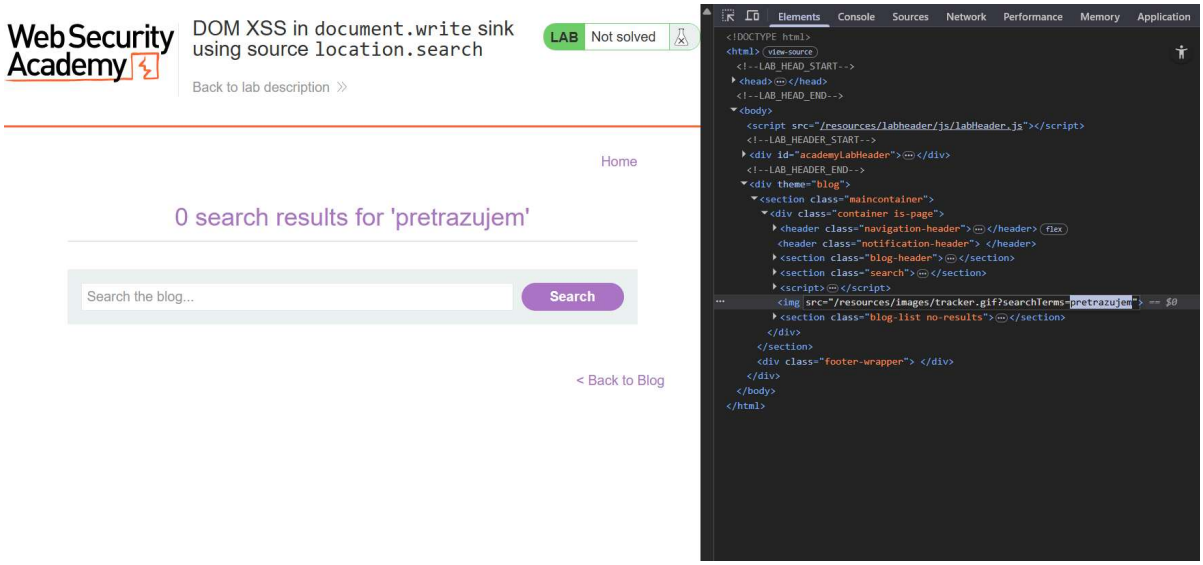
Lab 1 - DOM XSS in document.write sink using source location.search

The goal of this lab was to exploit a DOM-based XSS vulnerability where user input from the URL query string is passed directly into a `document.write()` call, which writes the value into an `img` tag's `src` attribute without any sanitization.

Steps

Step 1 - Injecting a Test String into the Search Box

A random alphanumeric string was entered into the search box. Using browser DevTools (Inspect Element), it was observed that the string was placed directly inside an `img` tag's `src` attribute in the DOM, without any encoding or sanitization. This confirmed that the `location.search` source flows directly into a `document.write()` sink.



Step 2 - Breaking Out of the img Attribute

Since the input landed inside a `src` attribute, the goal was to break out of that attribute and inject executable HTML. The following payload was entered into the search box:

```
"><svg onload=alert(1)>
```

The `"` closes the `src` attribute value, `>` closes the `img` tag, and the `<svg onload=alert(1)>` injects a new SVG element that executes JavaScript as soon as it loads.

Step 3 - Alert Triggered

After submitting the payload, the browser executed the injected JavaScript and an alert dialog appeared, confirming successful exploitation of the DOM XSS vulnerability.

The application uses `document.write()` to dynamically write content to the page, incorporating data directly from `location.search` without any sanitization or encoding. This allows an attacker to inject arbitrary HTML and JavaScript by simply crafting a malicious URL with a payload in the query string.

The `document.write()` function is a dangerous sink that should be avoided, especially when used with data from attacker-controllable sources such as `location.search`. User input must always be sanitized and encoded before being inserted into the DOM. A safer alternative is to use `textContent` instead of `innerHTML` or `document.write()`, as it treats input as plain text and does not interpret HTML or JavaScript.

🟢 Lab 2 - DOM XSS in jQuery anchor href attribute sink using location.search source

The goal of this lab was to exploit a DOM-based XSS vulnerability where jQuery reads a URL parameter and assigns it directly to an anchor tag's `href` attribute, allowing injection of a `javascript:` URL that executes arbitrary code when clicked.

Steps

Step 1 - Identifying the Vulnerable Parameter

By navigating to the submit feedback page, it was observed that the URL contained a `returnPath` parameter with the value `/`. This parameter is used by jQuery to dynamically set the `href` attribute of the back link on the page. Since the value comes directly from `location.search` and is passed into the `href` attribute without sanitization, this is a classic taint flow from source to sink.

Step 2 - Injecting the Payload

The `returnPath` parameter in the URL was changed from `/` to:

```
javascript:alert(document.cookie)
```

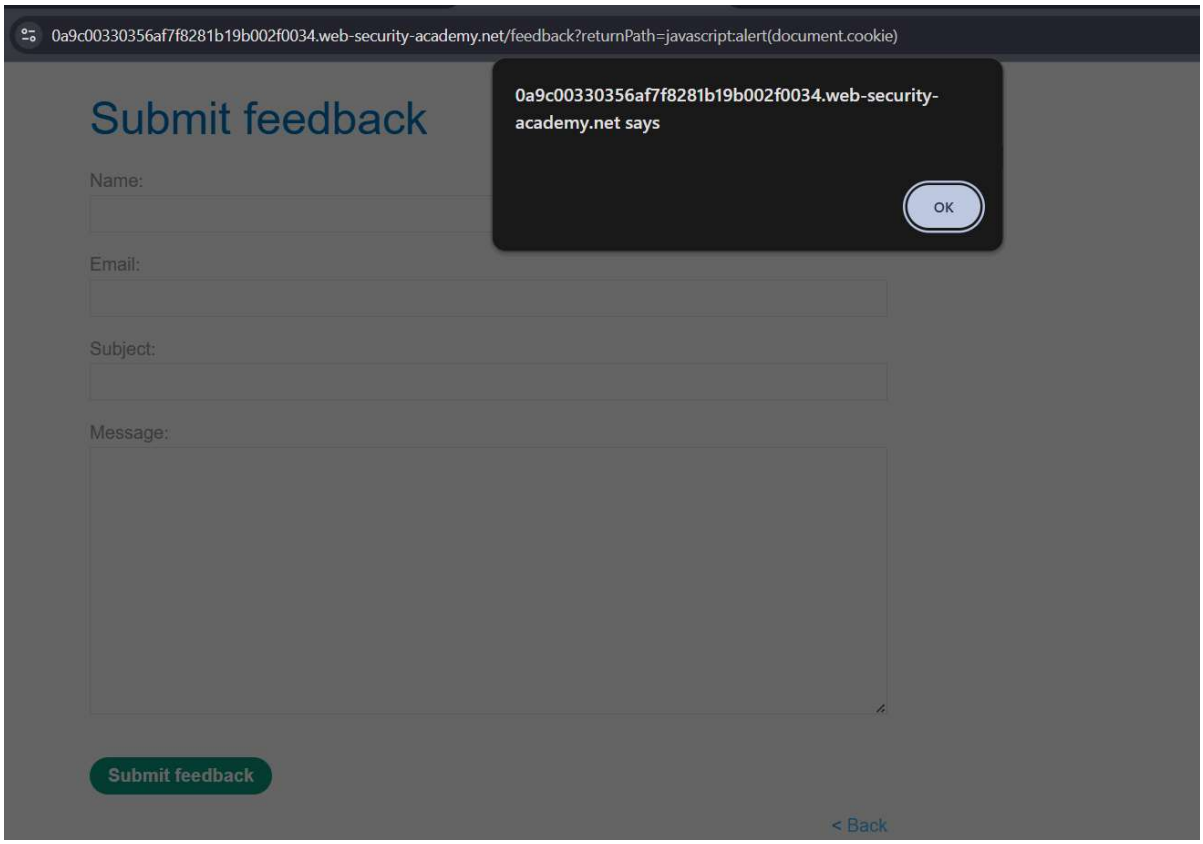
The modified URL looked like this:

```
/feedback?returnPath=javascript:alert(document.cookie)
```

This caused jQuery to set the `href` attribute of the back link to the injected `javascript:` URL, meaning that clicking the link would execute JavaScript instead of navigating to a page.

Step 3 - Triggering the Alert

After clicking the back link, the browser executed the injected JavaScript and an alert dialog appeared displaying the current session cookie, confirming successful exploitation.



The application uses jQuery to read the `returnPath` parameter from `location.search` and assigns it directly to an anchor tag's `href` attribute. Since `href` accepts `javascript:` URLs, an attacker can inject arbitrary JavaScript that executes when the link is clicked. No validation or sanitization was applied to the parameter before it was passed to the sink.

Anchor `href` attributes are dangerous sinks when populated with user-controlled data, because browsers support `javascript:` URLs which execute code on click. User input used to set `href` values must be validated to only allow safe URL schemes such as `https://`. jQuery's `attr()` method does not provide any protection against this — validation must be implemented explicitly in the application code.

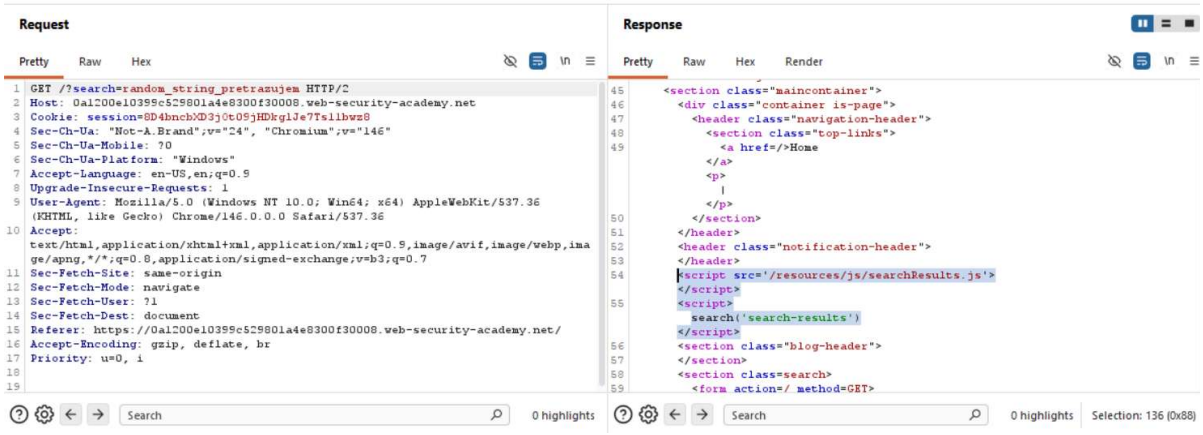
Lab 3 - Reflected DOM XSS

The goal of this lab was to exploit a reflected DOM XSS vulnerability where the server echoes user input in a JSON response, which is then processed by a client-side script using the dangerous `eval()` function, allowing injection of arbitrary JavaScript.

Steps

Step 1 - Intercepting the Search Request

With Burp Suite Intercept enabled, a random test string was entered into the search bar. The request was forwarded and it was observed that the search term was reflected directly in a JSON response from the endpoint `search-results`.



Step 2 - Identifying the Vulnerable Code

By examining the `searchResults.js` file from the Site Map, it was found that the JSON response is passed directly into an `eval()` function call. This means whatever the server reflects in the JSON response gets executed as JavaScript code on the client side.

Additionally, through testing different inputs it was observed that the application escapes double-quote characters (`"`) but does not escape backslashes (`\`), which is the key to the exploit.

searchResults.js

```javascript
function search(path) {
    var xhr = new XMLHttpRequest();
    xhr.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            eval('var searchResultsObj = ' + this.responseText);
            displaySearchResults(searchResultsObj);
        }
    };
    xhr.open("GET", path + window.location.search);
    xhr.send();

    function displaySearchResults(searchResultsObj) {
        var blogHeader = document.getElementsByClassName("blog-header")[0];
```

Step 3 - Crafting the Payload

The following payload was entered into the search box:

```
\"-alert(1)}//
```

Here is how the payload works step by step:

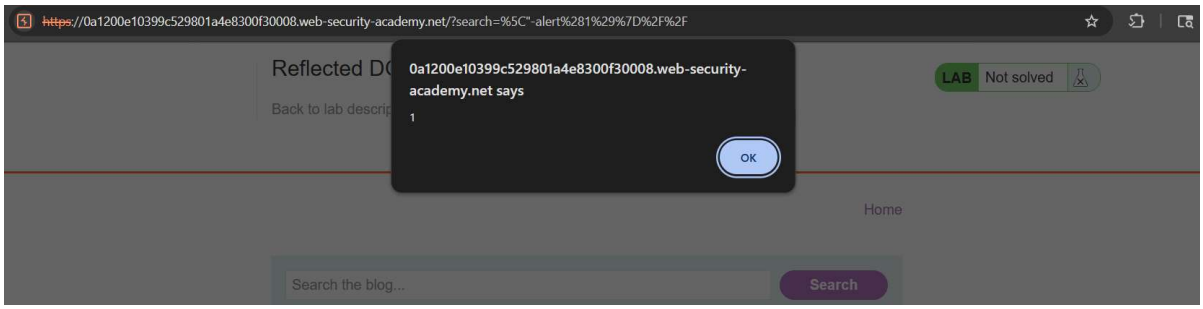
- The `\` is injected as a backslash. Since the server escapes `"` by adding a `\` before it, the result is `\\"` — a double backslash, which cancels out the escaping and leaves the `"` unescaped, effectively closing the JSON string.
- The `-` arithmetic operator separates the two expressions.
- `alert(1)` is the injected function call.
- `}` closes the JSON object early.
- `//` comments out the rest of the original JSON response.

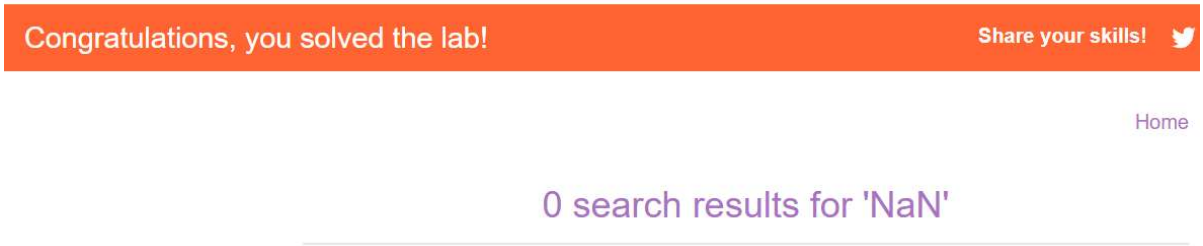
The final JSON response processed by `eval()` looked like this:

```
{"searchTerm":"\\"-alert(1)}//", "results":[]}
```

Step 4 - Alert Triggered

After submitting the payload, the browser executed the injected JavaScript and an alert dialog appeared. The page also displayed `0 results for 'NaN'`, which confirmed that the JSON parsing was disrupted by the payload and the arithmetic expression was evaluated instead of a search term.





The server reflects user input directly into a JSON response without fully sanitizing it — backslashes are not escaped, only double quotes are. The client-side script then passes this JSON response to `eval()`, which executes the content as JavaScript. This combination of incomplete server-side sanitization and a dangerous client-side sink creates a reflected DOM XSS vulnerability.

The `eval()` function is one of the most dangerous sinks in JavaScript and should never be used to process data that originates from user input or server responses that include user-controlled values. All special characters, including backslashes, must be properly escaped before being included in JSON responses. On the client side, JSON should always be parsed using `JSON.parse()` instead of `eval()`, as `JSON.parse()` does not execute code.

🔵 Lab 4 - Stored DOM XSS

The goal of this lab was to exploit a stored DOM XSS vulnerability in the blog comment functionality, where the application attempts to sanitize user input using JavaScript's `replace()` function, but does so in an incomplete way that can be bypassed.

Steps

Step 1 - Identifying the Vulnerability

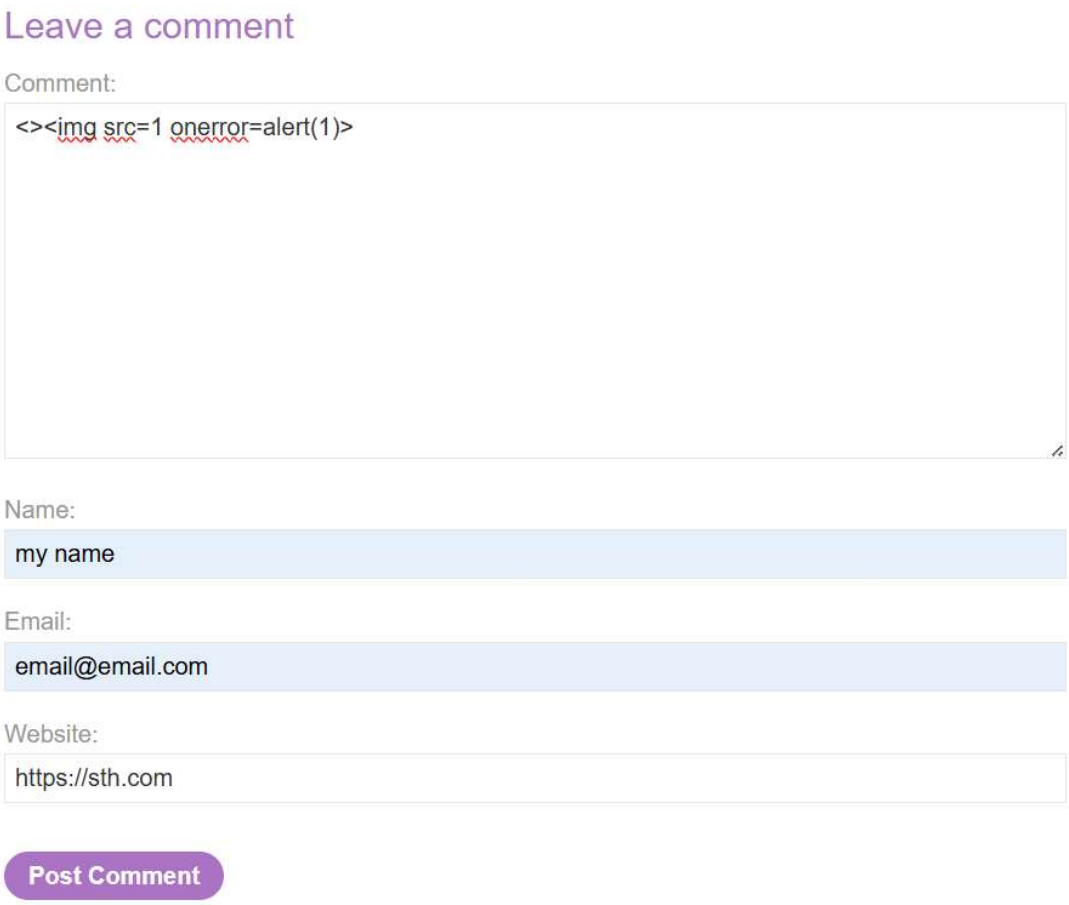
The blog comment section was identified as the target. The application uses JavaScript's `replace()` function to encode angle brackets (`<` and `>`) in an attempt to prevent XSS. However, when `replace()` is called with a string as the first argument instead of a regular expression, it only replaces the **first occurrence** of that character, leaving any subsequent angle brackets untouched.

Step 2 - Crafting the Payload

To bypass the filter, the following payload was posted as a comment:

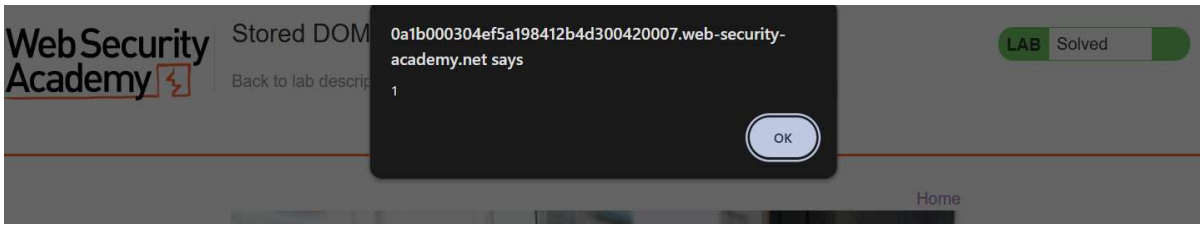
```
<><img src=1 onerror=alert(1)>
```

The trick is the empty `<>` at the beginning. The `replace()` function encodes only those first angle brackets, consuming the one replacement it will perform. All the angle brackets that follow in `<img src=1 onerror=alert(1)>` are left completely unencoded and are interpreted as raw HTML by the browser.



Step 3 - Alert Triggered

When the blog post page was loaded, the stored comment was retrieved and processed by the vulnerable JavaScript. The browser interpreted the unencoded `<img>` tag, attempted to load an image from the invalid source `1`, and since that failed, the `onerror` event handler fired and executed `alert(1)`.



The application tried to sanitize user input by replacing angle brackets using JavaScript's `replace()` function with a string argument. Since `replace()` with a string only replaces the first match, an attacker can bypass this filter by prepending a decoy `<>` at the start of the payload, which absorbs the single replacement, leaving the rest of the malicious HTML intact. The sanitized comment is then stored and later written into the DOM, where the browser executes the injected script.

JavaScript's `replace()` function must never be used for security-critical sanitization unless a global regular expression flag (`/g`) is used, as the string version only replaces the first occurrence. More importantly, user-generated content should never be written directly into the DOM without proper sanitization. A dedicated HTML sanitization library should be used, and dangerous sinks such as `innerHTML` should be avoided in favor of safer alternatives like `textContent`.

🔵 Lab 5 - DOM XSS in document.write sink using source location.search inside a select element

The goal of this lab was to exploit a DOM-based XSS vulnerability in the stock checker functionality, where user input from the URL query string is written directly into a `select` element using `document.write()`, allowing an attacker to break out of the select element and inject arbitrary HTML.

Steps

Step 1 - Identifying the Vulnerability

On the product page, the JavaScript extracts the `storeId` parameter from `location.search` and uses `document.write()` to create a new `option` element inside a `select` dropdown for the stock checker. A random alphanumeric string was added as the `storeId` parameter in the URL, and it was confirmed that the value appeared as a new option in the dropdown list. Inspecting the element confirmed that the input landed directly inside the `select` element without any sanitization.

Step 2 - Crafting the Payload

To break out of the `select` element and inject executable HTML, the URL was modified to include the following payload as the `storeId` parameter:

```
product?productId=1&storeId="></select><img%20src=1%20onerror=alert(1)>
```

The payload works as follows:

- `"` closes the `value` attribute of the `option` tag
- `>` closes the `option` tag

- `</select>` closes the entire select element, breaking out of it
- `<img src=1 onerror=alert(1)>` injects an image tag with an invalid source, causing the `onerror` event to fire and execute `alert(1)`

Step 3 - Payload Executed

After loading the modified URL, the injected HTML was written into the page via `document.write()`. The select dropdown was visibly disrupted, confirming that the HTML structure was broken out of successfully and the payload was interpreted by the browser.

The application reads the `storeId` parameter directly from `location.search` and passes it into `document.write()` to dynamically build a `select` element. Since no sanitization or encoding is applied, an attacker can inject arbitrary HTML by crafting a malicious URL. The injected content breaks out of the `select` element and introduces executable HTML into the page.

The `document.write()` function is a dangerous sink that should never be used with user-controlled data. Even when input is placed inside a structured HTML element like `select`, an attacker can break out of that context by injecting closing tags. User input must always be properly encoded before being inserted into the DOM, and safer DOM manipulation methods such as `createElement()` and `appendChild()` should be used instead of `document.write()`.